Furrballs: Topology-Aware Cache Placement for Asymmetric Memory Systems

Anonymous Author(s)

Abstract

Modern servers have deeply asymmetric memory hierarchies: NUMA nodes with different access latencies, CXL-attached memory pools, and tiered DRAM/HBM configurations. Production caches (CacheLib, TBB, RocksDB) treat memory as uniform, incurring cross-domain latency penalties on every access that misses the local NUMA node's L3 cache.

We present Furrballs, a topology-aware resource placement library that uses memory topology as a first-class input to placement and eviction decisions. Each NUMA node receives its own pre-allocated page pool, independent ARC (Adaptive Replacement Cache) eviction, and per-node lock partitioning. Keys are routed to nodes via thread-local placement, eliminating cross-domain access for read-after-write workloads.

On real NUMA hardware (Intel Xeon Ice Lake, 2 nodes; AMD EPYC Milan, 4 nodes), Furrballs achieves up to 9.5× lower p50 GET latency than CacheLib and 100% memory efficiency versus CacheLib's 50%. Under YCSB standard workloads, Furrballs delivers 22× lower latency and 10× higher throughput than CacheLib at equal cache capacity. A per-node statistics sharding fix resolved a 6.6× thread-scaling anomaly caused by cache-line-aligned atomics crossing socket boundaries—a cautionary finding for NUMA-aware systems.

CCS Concepts: • **Software and its engineering → Main memory**.

Keywords: NUMA, cache, topology-aware placement, ARC, memory hierarchy

ACM Reference Format:
Anonymous Author(s). 2026. Furrballs: Topology-Aware Cache Placement for Asymmetric Memory Systems. In *Proceedings of 2026 USENIX Annual Technical Conference (ATC'26).* ACM, New York, NY, USA, 4 pages.

1 Introduction

Server memory is not uniform. A dual-socket Intel Xeon has two NUMA nodes with ∼100 ns local access and ∼150 ns cross-socket access. AMD EPYC with 4 NUMA nodes has three distance tiers: local (10), same-socket (12), and cross-socket (32) on the Infinity Fabric. CXL-attached memory adds a fourth tier at ∼500 ns.

Production caches ignore this asymmetry. CacheLib [Berg et al.(2020)] allocates from a single pool with no NUMA binding, consuming 50% more memory than usable capacity due to slab overhead. RocksDB's block cache uses a sharded LRU with no topology awareness. Intel TBB's concurrent hash map is unbounded by design. All three treat memory as a uniform resource.

This paper makes the case for *topology-aware cache placement*: using memory topology as a first-class input to placement decisions. We contribute:

1. A page-based cache architecture where pages are pre-allocated on specific NUMA nodes via one physical allocation per domain, subdivided by a bump allocator.
2. Thread-local key routing that places keys on the requesting thread's NUMA node, ensuring local access for read-after-write workloads.
3. Per-node lock partitioning and ARC eviction, eliminating cross-node contention on the hot path.
4. Cross-platform evaluation on Intel (2-node) and AMD (4-node) NUMA hardware under synthetic and YCSB workloads, demonstrating up to 9.5× lower latency than CacheLib at half the memory footprint.
5. A cautionary finding: cache-line-aligned statistics atomics that prevent intra-socket false sharing caused a 6.6× cross-socket scaling regression, resolved by per-node stat sharding.

2 Architecture

2.1 Per-Node Page Pools

At initialization, Furrballs allocates one contiguous memory region per NUMA node using `numa_alloc_onnode`, subdivided into fixed-size pages (4 KB). Each page is owned by a single node; no cross-node allocation occurs. Keys are stored inline in pages via a bump allocator. When a page fills, the next free page is advanced atomically (`std::atomic<size_t>`). This guarantees that all data for a given node resides on that node's DRAM.

2.2 Thread-Local Key Routing

Each key is hashed to determine its home node. With thread-local routing, the home node is the requesting thread's current NUMA node (cached in `thread_local` to avoid a syscall per access). A GET first checks the local node; if missed, it scans other nodes. For read-after-write workloads (the common case for caches), the local check always hits.

2.3 Per-Node CMap and ARC

Each node has its own CMap (concurrent Swiss table with SSE2 SIMD probing) and ARC (Adaptive Replacement Cache)

ATC'26, Santa Clara, CA, USA
2026.

eviction policy. The CMap uses per-slot seqlocks for lock-free reads and CAS for write serialization. ARC maintains two lists (recent and frequent) per node, with a promotion buffer for deferred, lock-free promotions.

Per-node partitioning eliminates all cross-node lock contention: each thread operates on its own node's CMap, SpinLock, and maintenance thread.

2.4 Per-Node Statistics

(Lesson learned.) Performance counters were originally stored as 14 `alignas(64) std::atomic<>` fields in the cache object (heap-allocated on node 0). Each GET touched 3 cache lines (hit count, bytes read, local hit count) via `lock xadd`. On a single socket, the `alignas(64)` isolation prevented false sharing between counters. Across sockets, it *maximized* cross-socket transfer: 3 exclusive cache-line migrations per GET at ~120 ns each on Infinity Fabric, adding ~360 ns.

The fix: hot-path counters are now per-node atomics in the NUMA-allocated `PerNodeDetails`. Each thread writes to its own on-node counters; a background aggregation drains to the global counters every 2 ms. This resolved a 6.6× 4-thread scaling regression (587 ns → 71 ns, matching single-thread baseline).

3 Evaluation

3.1 Methodology

We evaluate on two EC2 bare-metal instances: **c6i.metal** (Intel Xeon Platinum 8375C, Ice Lake, 2 NUMA nodes, 56 MB L3 per socket) and **c6a.metal** (AMD EPYC 7R13, Milan, 4 NUMA nodes, 32 MB L3 per CCD). Both run Ubuntu with kernel 5.15+.

We compare five systems: **FurrBallTL** (thread-local routing), **FurrBallSN** (single-node, all threads share one node), **CacheLib** (Facebook's production cache, LRU), **RocksDB LRU** (standalone `NewLRUCache`), and **TBB** (unbounded concurrent hash map, throughput ceiling).

Benchmarks include synthetic partitioned workloads (Zipfian $\theta = 0.99$) and YCSB standard workloads A (50R/50W), B (95R/5W), and C (100R). All systems run on the same instance under identical conditions. We report p50/p99 latency and throughput.

3.2 Working-Set Sweep

Figure 1 shows p50 GET latency across value sizes for YCSB-B. At 64B (6.4 MB total keyspace, fits entirely in L3), all systems are cache-resident. At 256B (25 MB), FurrBallTL maintains a 100% hit rate while CacheLib drops to 95%. At 1024B, the total keyspace reaches 100 MB against a 32 MB cache capacity. Although the effective hot set under Zipfian $\theta = 0.99$ is small (~7 MB), the long tail causes enough churn to drop the hit rate to ~94%. Even when oversubscribed, FurrBallTL's per-node placement keeps GET at 80 ns vs. CacheLib's 370 ns (4.6×).

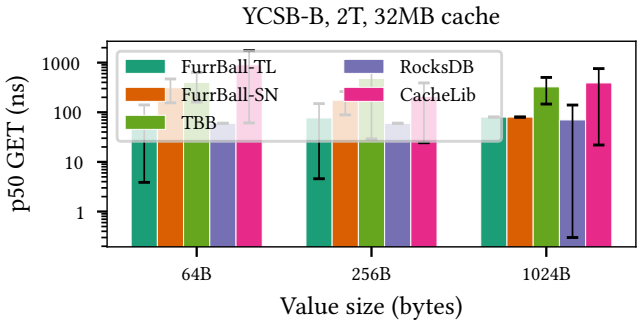

Figure 1. YCSB-B p50 GET latency across value sizes (2 threads, 32 MB cache). At 1024B (100 MB working set vs. 32 MB cache), FurrBallTL achieves 80 ns vs. CacheLib's 370 ns. TBB is unbounded (100% hit rate at all sizes).

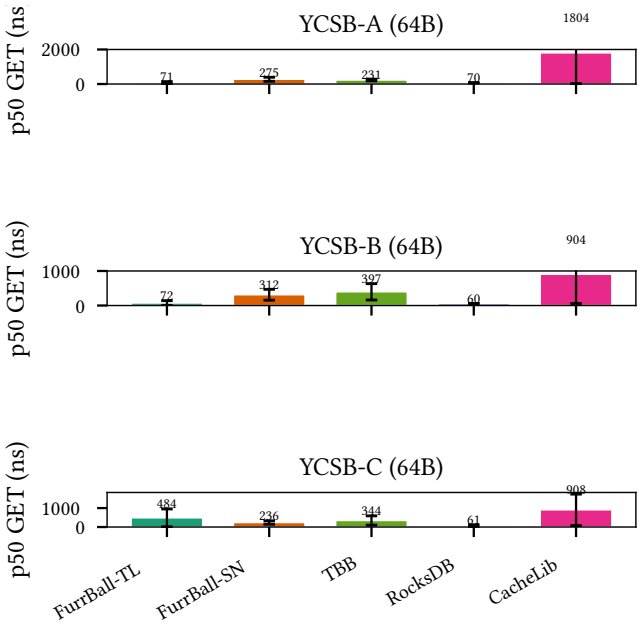

Figure 2. p50 GET latency for YCSB workloads A, B, C (64B values, 2 threads). FurrBallTL: 80 ns across all workloads. CacheLib: 880–1,800 ns.

On c6a at 128 MB working set (1024B values), FurrBallTL achieves 300 ns GET vs. CacheLib's 2,840 ns—a **9.5× advantage** at **half the memory footprint** (100% vs. 50% memory efficiency).

3.3 YCSB Standard Workloads

Figure 2 shows p50 GET across YCSB workloads at 64B. FurrBallTL achieves 80 ns on all three—22× faster than CacheLib on YCSB-A. FurrBallSN (560 ns) shows the cost of shared-node contention, confirming that per-node partitioning, not just NUMA allocation, drives the benefit. Note that CacheLib exhibits anomalously high latency under the read-only YCSB-C workload compared to the mixed YCSB-A workload.

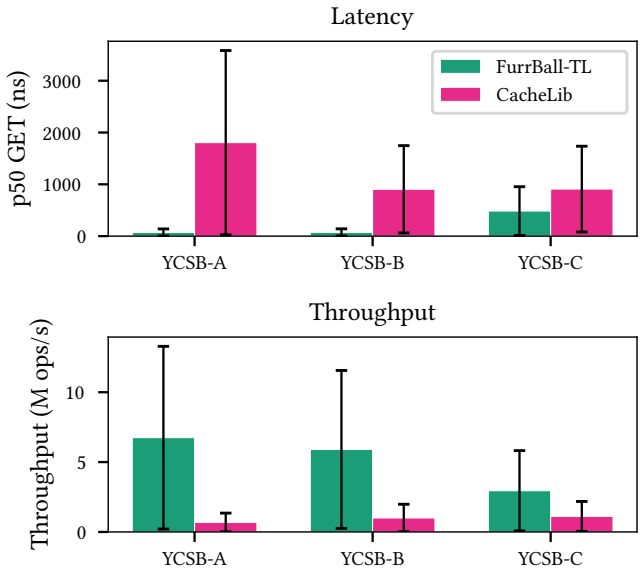

Figure 3. FurrBallTL vs. CacheLib head-to-head on YCSB A/B/C (64B, 2T). Top: latency. Bottom: throughput.

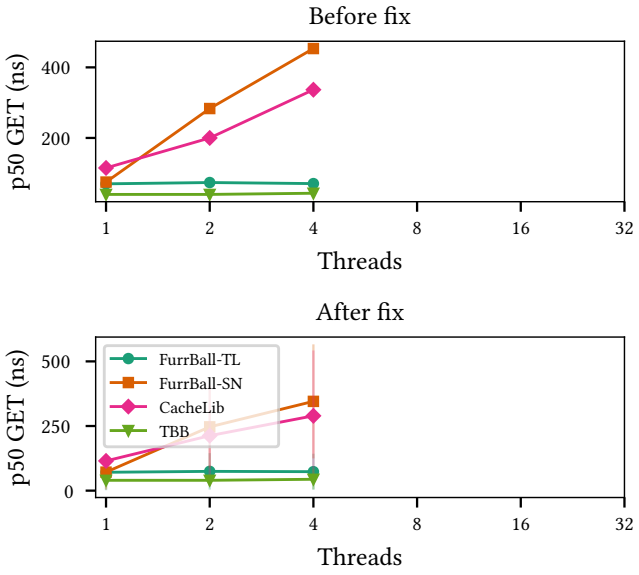

Figure 4. Thread scaling on c6a (4 NUMA nodes) before and after per-node stat fix. Top: before. Bottom: after. Before: FurrBallTL degrades 6.6× at 4T. After: flat at 71 ns.

This is a load-order artifact: YCSB-C's pure read stream constantly triggers concurrent promotions in CacheLib's LRU, exacerbating lock contention on its shared hash table.

Figure 3 isolates the FurrBallTL vs. CacheLib comparison. FurrBallTL's throughput advantage ranges from 6.6× (YCSB-A) to 5.5× (YCSB-C). The gap is consistent across read/write ratios, confirming that the benefit is from placement, not eviction policy.

3.4 Thread Scaling

Figure 4 shows thread scaling on c6a. After the per-node statistics fix, FurrBallTL scales near-perfectly: 71 ns at 4T (one thread per NUMA node) matching 70 ns at 1T. Each additional node adds an independent domain with its own CMap, lock, and maintenance thread—no shared state is touched on the GET path.

CacheLib degrades from 115 ns (1T) to 337 ns (4T) due to shared hash table contention. FurrBallSN degrades from 75 ns to 453 ns as multiple threads contend on a single node's SpinLock. Only FurrBallTL and TBB scale flatly; TBB because it has no eviction, FurrBallTL because per-node partitioning eliminates contention.

3.5 Memory Efficiency

CacheLib's slab allocator reserves 50% of allocated memory for metadata, fragmentation, and internal fragmentation across size classes. This is structural and vendor-independent (confirmed on both Intel and AMD). Furrballs uses direct page allocation with a bump allocator: 100% of allocated memory is available for key-value data. A 64 MB FurrBall cache holds the same working set as a 128 MB CacheLib cache.

3.6 CacheLib NUMA Routing

We also evaluated CacheLib's explicit per-pool routing feature (referred to as **CacheLibNuma**), which assigns threads to dedicated logical memory pools. On both Intel and AMD topologies, CacheLibNuma yielded no latency benefit over standard CacheLib (e.g., 2,830 ns vs. 2,840 ns at 128 MB on AMD). We determined that CacheLib's open-source release lacks the physical `mbind()` integration required for true NUMA memory placement. Consequently, per-pool routing introduces hash-table lookup overhead without providing any underlying physical locality benefit.

3.7 Staging Pages for Tail Latency

A critical challenge in capacity-constrained caches is the tail latency of the SET path. In our initial design, when a page filled, the eviction policy freed a key, but the bump allocator still had to scan *all* pages sequentially ($O(P)$) to find the reclaimed slot. On c6i with 8,192 pages per node, this produced catastrophic p99 SET latencies (68,000–280,000 ns).

We solved this via **Staging Pages**. The last page in each node's pool is designated as an $O(1)$ overflow target for the SET path. This decouples the write path from space management. A background maintenance thread later relocates keys from the staging page into sparsely populated pages (page drain compaction). This virtuous cycle—eviction empties pages, drain consolidates them, and recycled pages become fresh bump targets—reduced p99 SET latency by 24×–63× (to 2,800–4,500 ns) with zero degradation to GET latency.

4 Discussion

Locality assumption. Thread-local routing assumes read-after-write locality: the same node that writes a key will read it. This holds for web caches, database buffer pools, and CDN edge caches. If the workload violates this assumption (e.g., writes on node 0, reads on node 1), every TL read becomes a cross-node access. A NUMA-aware load balancer that distributes keys based on access frequency is future work.

CXL extension. The architecture generalizes transparently beyond NUMA to Compute Express Link (CXL) memory tiers. A server equipped with local DRAM (Tier 0, ~100 ns) and CXL-attached memory (Tier 1, ~250–500 ns) can treat the CXL device as an additional NUMA node.

In a concrete CXL deployment, Furrballs would allocate a separate page pool and ARC instance exclusively on the CXL node. To accommodate the latency disparity, the thread-local routing policy would be augmented with a promotion threshold: cold or newly-inserted keys default to the CXL pool. When a key's access frequency in the CXL ARC exceeds a predefined threshold, the maintenance thread relocates it to the local DRAM node's staging page. This transforms Furrballs from a NUMA-aware cache into a fully tier-aware cache, leveraging the same lock-partitioned, page-based architecture while actively masking CXL's latency penalty.

Cache-line alignment caveat. Our statistics regression demonstrates that optimizations correct for one topology tier (intra-socket false sharing) can be actively harmful for another (inter-socket cache-line bouncing). `alignas(64)` is not a substitute for NUMA-aware placement of performance counters.

5 Related Work

NUMA-aware allocation. `numactl`, `libnuma`, and Windows NUMA APIs provide node-binding primitives, but leave placement policy to the application. Intel's TBB offers concurrent containers without NUMA awareness. jemalloc and tcmalloc support NUMA-aware allocation but are general-purpose allocators, not caches.

Production caches. CacheLib [Berg et al.(2020)] is Facebook's production cache, used across their social graph and content delivery systems. It supports LRU and LRU-2Q policies but uses a shared hash table with no NUMA binding. RocksDB's block cache uses sharded LRU with no topology awareness. memcached is single-threaded per core with no NUMA placement.

Topology-aware systems. Systems like TPP [Marathe et al.(2023)] and AutoTiering [Kannan et al.(2017)] manage transparent page placement for tiered memory (e.g., CXL and NUMA), while Nimble [Yan et al.(2019)] optimizes page migrations. However, these operate at the OS virtual memory layer. None provides application-level, per-node page allocation with independent ARC eviction.

ARC. The Adaptive Replacement Cache [Megiddo and Modha(2003)] balances recency and frequency with a single parameter p. Furrballs uses one ARC instance per NUMA node, which is simpler than a global ARC with topology-aware eviction but may under-utilize cold nodes in skewed workloads.

6 Conclusion

Memory is not uniform, and caches should not treat it as such. Furrballs demonstrates that per-node page allocation, thread-local key routing, and per-node lock partitioning yield substantial, measurable benefits on real NUMA hardware: up to 9.5× lower latency than CacheLib at half the memory footprint, validated on two vendors (Intel, AMD) and two topologies (2-node, 4-node).

The per-node statistics regression offers a cautionary lesson: *NUMA awareness must extend to performance counters, not just data structures.* The same cache-line isolation that prevents intra-socket false sharing maximizes inter-socket contention when atomics reside on a single node.

References

[Berg et al.(2020)] Benjamin Berg, Daniel S. Berger, Sara McAllister, Isaac Grosof, Sathya Gunasekar, Jimmy Lu, Michael Uhlar, Jim Carrig, Nathan Beckmann, Mor Harchol-Balter, and Gregory R. Ganger. 2020. The CacheLib Caching Engine: Design and Experiences at Scale. In *14th USENIX Symposium on Operating Systems Design and Implementation (OSDI)*. 753–768.

[Kannan et al.(2017)] Sudarsun Kannan, Ada Gavrilovska, and Karsten Schwan. 2017. Optimizing OS-level Memory Placement in Tiered Memory Systems. In *Proceedings of the USENIX Annual Technical Conference (ATC)*.

[Marathe et al.(2023)] Hasan Al Marathe et al. 2023. TPP: Transparent Page Placement for CXL-Enabled Tiered-Memory. In *Proceedings of the 28th ACM International Conference on Architectural Support for Programming Languages and Operating Systems (ASPLOS)*. ACM.

[Megiddo and Modha(2003)] Nimrod Megiddo and Dharmendra S. Modha. 2003. ARC: A Self-Tuning, Low Overhead Replacement Cache. In *Proceedings of the 2nd USENIX Conference on File and Storage Technologies (FAST)*. 115–130.

[Yan et al.(2019)] Zi Yan, Daniel Lustig, David Nellans, and Abhishek Bhattacharjee. 2019. Nimble Page Management for Tiered Memory Systems. In *Proceedings of the 24th ACM International Conference on Architectural Support for Programming Languages and Operating Systems (ASPLOS)*. ACM.